

HUDDLE

Progressive Web Application

Software Engineering Systems Report

Software Engineering

Tempe High School

Michael Liondis

1. Identifying and Defining

1.1 Define and Analyse Problem Requirements

Problem Context

Huddle is a Progressive Web App (PWA) designed to make it easier for friend groups to organize and commit to social plans by providing a simple platform where users can propose activities, vote on them, and confirm attendance without relying on messy group chats or inconsistent responses.

Friend groups often struggle to coordinate plans due to:

- Low engagement in group chats (ignored messages or slow replies)
- Indecision when choosing activities
- Difficulty finding times that suit everybody
- Plans falling through due to lack of commitment

This can lead to immense frustration - especially if you are the one trying to plan something - and fewer group activities. The app will be designed for informal social groups such as friends, classmates, or clubs who want a faster and clearer way to make group decisions.

Needs and Opportunities

Need	Description
1. User Authentication & Group Management	Users must be able to securely create accounts, log in with Google Authenticator 2FA, and create or join friend groups via a unique alphanumeric invite code.
2. Event Proposal & Voting	Users must be able to propose events with a name, date/time, and location, and group members must be able to cast Yes/No/Maybe votes with live percentage tallies displayed in real time.
3. Attendance Tracking & Role Assignment	The system must track who has committed to attending an event, allow custom roles (e.g. 'Bringing snacks', 'Ride home') to be assigned to members, and support preset role options via a dropdown.

SWOT Analysis

Strengths:

- It solves a painful point that is highly relatable to my target audience
- By explicitly excluding things like real time chats and payment systems, the software engineering project stays manageable for a student project.
- By using a PWA it ensures cross-platform compatibility (iOS, Android, laptop) without the complexity of separate app store submissions.
- Familiarity with Secure PWA Architecture significantly reduces the research needed in the development phase.

Weaknesses:

- The app is only useful if an entire friend group joins; getting one person to switch is easy but getting 5 - 10 people is hard.
- Relying on google authenticator adds security but also creates a barrier for entry. If a simple-minded user doesn't have the app or finds the 2FA process too "heavy" for a casual social plan, they may abandon the sign-up.
- Since this relies heavily on a central SQL database and backend server, any downtime or hosting issues completely breaks the Huddle web app for all users

Opportunities:

- Could potentially be used by corporate companies trying to find a modern and fun solution to organising workflows or assign fun team building tasks towards a team.
- Beyond friend groups, this is perfect for school clubs or sports teams that need to organize attendance tracking but don't need a full management suite.
- Successfully deploying a functional PWA with a backend and SQL database is a massive real-world achievement for my portfolio.
- If the core voting aspect is a success and is enjoyed by the community, implementations such as "Quick Polls" can be implemented for things other than events, e.g. "What should we eat?" or "Who is taking the train?"

Threats:

- People are heavily addicted to the likes of WhatsApp, Discord, Instagram, or iMessage. Moving them to a separate app for potentially only one specific task is a big hurdle.
- Introducing something new into a group of people who are already used to a different application might turn the app into a 'one-hit-wonder' where members think the gimmick is cool and useful but forget about it and move back to what they were using before quickly.
- Without a native apps push notification, users might forget to check the app to vote, leading back to one of the original problem of "Ignored messages"

Boundaries

Operational & time:

- The system must be completed within the school term, meaning high complexity features are outside the boundary
- While designed for friend groups, the system may face performance boundaries if a single 'group' exceeds a certain size (e.g. 100+ members voting together)

Operating system & software boundaries:

- The system will rely heavily on google authenticator. If that device is down or a user loses access to their 2FA device/email, they are locked out of their Huddle.
- Unlike native apps, Huddle cannot access deep OS features like accessing contact lists or hardware-level alarm systems.
- The app is limited by the capabilities of modern browsers (Chrome, Safari, Firefox). Features may behave differently on iOS vs. Android

Hardware boundaries:

- The system must remain functional over varying screen size: Smartphone to PC
- As a PWA, the app must be lightweight enough to run on old tech without lag
- The system requires an active internet connection to sync votes and event data

Security & data boundaries:

- The system is bound by the structure of an SQL database. It cannot handle unstructured data (social media posts) outside of the predefined event fields.
- Information is bound by "Friend Groups" - users in group A cannot see events or participations in group B unless they have a specific invite code

Social & Ethical Issues

Privacy - The ethics of data collection

Many social apps over-collect data (e.g. location, contacts, or browsing habits), which can lead to surveillance creep where users feel tracked.

To address this issue, Huddle only collects the bare minimum data for function: an email address and a display name. Unlike other native apps, the PWA structure means the app cannot access deep OS features like user's contact list. This will create a privacy-first environment where friends can coordinate without the fear of their personal data being harvested or sold. Furthermore, the SQL database boundaries ensure the data is siloed: What happens in "friend group A" is entirely invisible to "friend group B", protecting the private social habits of different circles.

Security - Responsibility for user safety

A data breach or account takeover doesn't just leak the data; it can lead to physical safety risks as well (e.g., data breacher knowing exactly when and where a group of students will be meeting).

As a developer, I have an ethical responsibility to secure user coordination. To mitigate the risks of unauthorized access, Huddle will be integrated with Google Authentication Two-Factor-Authentication (2FA). Even if a user's password is leaked, the 2FA requirement acts as a critical secondary barrier. Additionally, the invite code system acts as a security boundary. Because there is no 'public search' for groups, a malicious actor cannot find or join a huddle without a direct human-sent link.

Accessibility - Inclusivity in the software design

Many software projects often exclude users by ignoring diverse physical or technological limitations, preventing individuals with disabilities from navigating their platforms.

Huddle addresses this by continuously adhering to the global Web Content Accessibility Guidelines (WCAG) accessibility standards. The PWA layout will scale dynamically across smartphones, laptops, and desktop computers. This responsive design ensures high quality readability and clear navigation on all screen sizes. Furthermore, the lightweight code runs smoothly on older hardware without causing performance lag. This prevents economic barriers from restricting user participation and social planning.

Environmental - reducing digital carbon footprint

Inefficient software and bloated cloud applications generate massive carbon emissions by consuming excessive electricity from server hosting. Bloated software increases server processing workloads, which drives up electrical consumptions and cooling needs.

Huddle addresses this environmental concern through its architectural and scope boundaries: by selecting a structured SQL database over unstructured alternatives, the system minimises data processing overhead. The decision to exclude high-complexity features such as social-media feeds and real-time chat reduces continuous server hosting. This lightweight approach limits the energy required for hosting and lowers the project's overall digital carbon footprint.

Intellectual property - Ethical use of shared code

Unauthorised use of open-sourced code violates copyright laws and exploits community labour, potentially creating severe legal liabilities for software projects. Developers face an ethical obligation to respect the intellectual properties of others by software license.

Huddle utilises open-sourced components for its users interface layout, backend server framework, and SQL database connectors. The project maintains an inventory of third-party libraries and complies with licencing terms

Functional Requirements

Requirement	Specification	Priority
Key voting/counting feature that allows users to select a preference on a proposed plan	System must tally “Yes”, “No”, “Maybe” votes; display real time percentage of a groups consensus.	Must
Event Proposal system allowing users to input event details	Input fields for Event Name (max 30 chars); Date/Time picker; Location (max 100 chars)	Must
User authentication via Google Authenticator 2FA for secure login	Requires a 6-digit time-based code (Google Authenticator TOTP) after password entry.	Must
Group creation/joining via a unique alphanumeric invite code	Generates a 6-digit random code; database must link new UID to the specific group id.	Must
A feature where you can assign roles to the attending members such as “food bringer” or “ride home” to let others know who is bringing what or how people are getting home.	Input bar accepts text input; minimum 3 characters; maximum 50 characters;	Must
Ability to save popular roles such as “ride home” or “bringing snacks” depending on popularity within the application	Drop down menu next to the search bar that holds pre-selectable answers; must be approved by dev to reduce profanities;.	Must
Attendance Tracker to display a list of committed members	Visual list “Confirmed” users; update status immediately upon “yes” or “no” vote	Nice
Interactive event map that pings the location of the hangout	Integration with map python library; display a single marker (ping) based on the event's address string.	Nice

Non-Functional Requirements

Requirement	Specification	Priority
Availability	The PWA must be accessible to users 99.5% of the time during peak social hours (9 a.m. - 10 p.m.)	Must
Responsiveness	The voting tally must update and be visible to all group members within 2 seconds of a vote being cast	Must
Data Security	All users and group information must be encrypted	Must

Requirement	Specification	Priority
Compatibility	The interface must be able to render correctly on screens ranging from 320px (mobile) to 1920px (desktop)	Must
Usability	A first-time user must be able to create a group and propose an event in under 3 minutes without a tutorial	Nice
Scalability	The backend server and SQL database must support up to 50 concurrent users/group members without lag	Nice

1.2 Tools to Develop Ideas and Generate Solutions

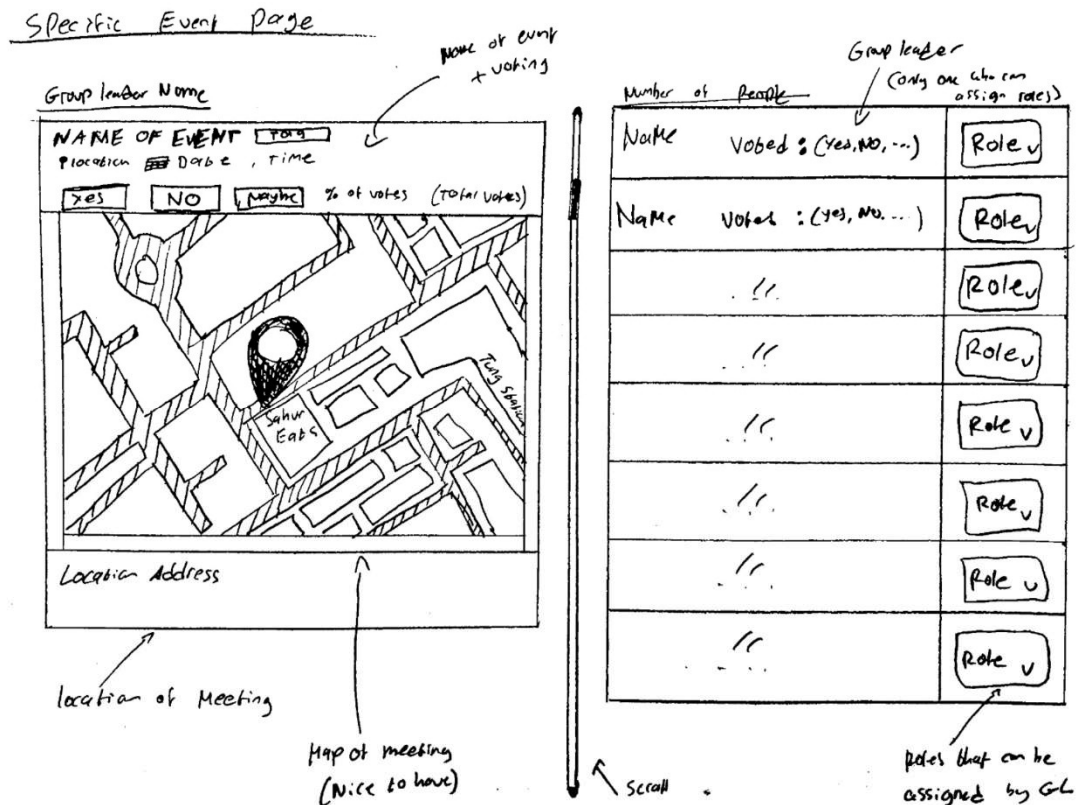
Brainstorm / Feature Assessment Table

The following table is a brainstorm table used to ideate and explore possible features, platforms, and design decisions for Huddle. Each idea was assessed for its pros and cons before being classified as a core requirement, a nice-to-have, or out of scope (with reference to client). This process directly shaped the functional and non-functional requirements documented in Section 1.1.

Category	Idea / Feature	Pros	Cons / Risks
Core Feature	Yes/No/Maybe voting with live tally	Clear group consensus; simple UX	Real-time sync adds complexity
Core Feature	Event proposal form (name, date, location)	Structured input reduces ambiguity	Location field needs geocoding for map
Core Feature	Group creation and invite codes	Private groups; no public search risk	User must share code out-of-band
Security	Google Authenticator 2FA on login	Strong secondary barrier against account takeover	Adds friction; lock-out risk if device lost
Platform	Progressive Web App (PWA)	Cross-platform; no app store required	No deep OS access (contacts, native push)
Nice-to-Have	Interactive Leaflet.js map on event detail	Informed voting; professional feel; no external app	Requires geocoding integration; JS complexity
Nice-to-Have	Attendance tracking post-event	Commitment visibility; closes the loop after event	Requires post-event notification logic
Out of Scope	Real-time group chat	Familiar for users	Recreates the problem Huddle is solving; too complex
Out of Scope	Calendar sync / scheduling assistant	Powerful scheduling	High complexity; out of scope for school term

Storyboard: Event Details Page

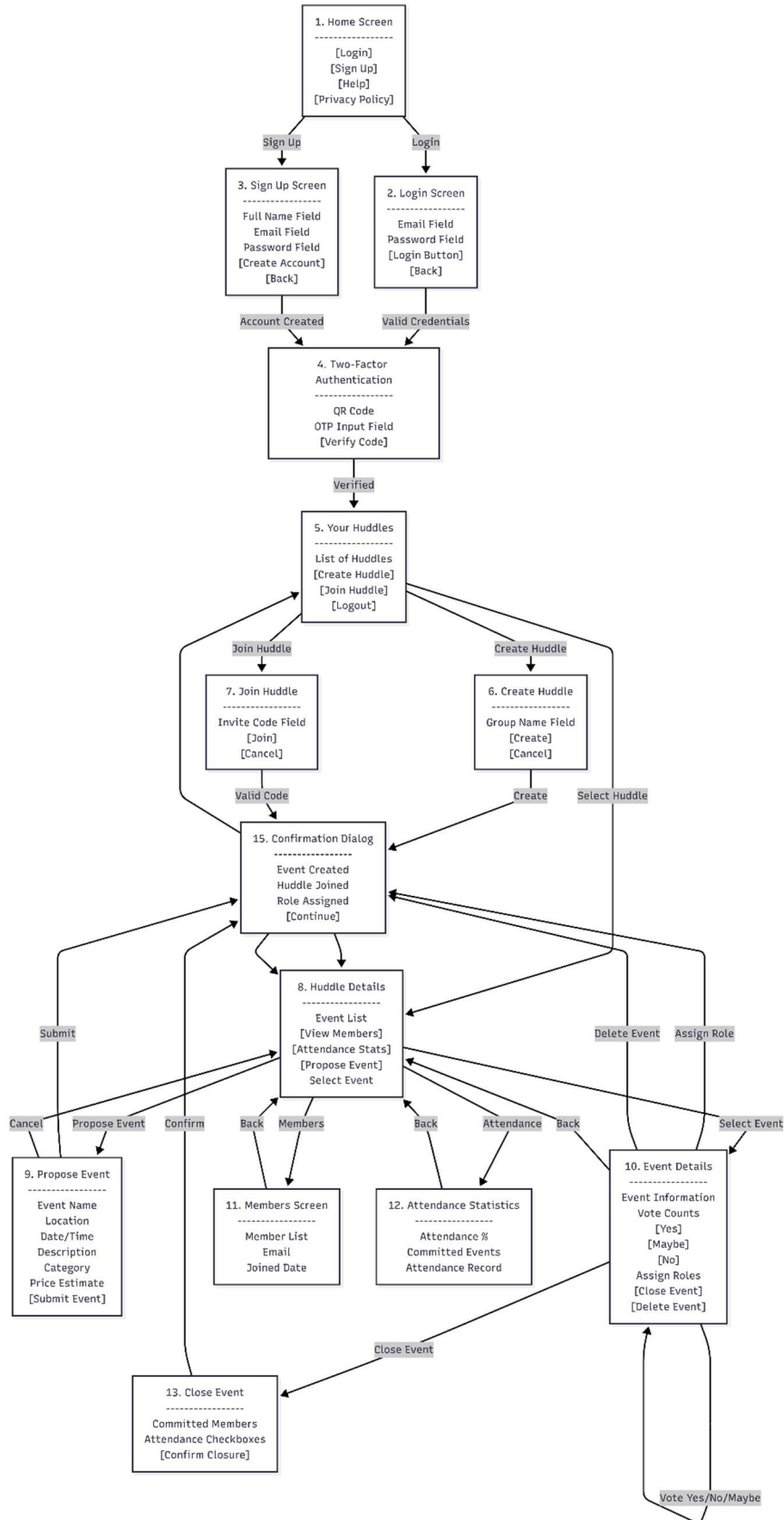
The storyboard below was developed in collaboration with the primary client during the requirements phase. It represents the intended layout and functionality of the Specific Event Detail Page - the central screen where group members view event details, cast their Yes/No/Maybe vote, see the interactive map, and view the attendance and role list.



The storyboard shows two panels. The left panel depicts the event detail view, including the event name, a Tag label, event location and date/time, Yes/No/Maybe voting buttons with a live vote percentage and total count, and an embedded interactive map with a location pin. The group leader's name is displayed at the top. The right panel shows a scrollable member list with each member's name, current vote, and a Role dropdown button. Only the group leader can assign roles to attending members.

The client specifically requested a storyboard for this page due to the complexity of its layout, as it functions as the enclosure holding most of the key features.

Storyboard: All pages



1.3 Implementation Method

Direct Cutover Approach

I have chosen the Direct Cutover Approach because Huddle is a new system that does not have a direct digital predecessor to run alongside.

To manage the risks of a direct launch, I will be conducting extensive beta testing with primary stakeholders/clients throughout the development. This ensures that when the 'cutover' occurs, the software will already be proven to meet the user's needs.

1.4 Financial Feasibility

SWOT Analysis

Strengths	Weaknesses
• Solves a highly relatable pain point for the target audience. • Explicit out-of-scope decisions keep the project manageable. • PWA ensures cross-platform compatibility without separate app store submissions. • Familiarity with secure PWA architecture reduces research time in development.	• App is only useful if an entire friend group joins - onboarding multiple people is hard. • Google Authenticator 2FA adds security but creates a barrier for casual users. • Dependency on a central SQL database and backend means any downtime breaks the entire app.
Opportunities	Threats
• Potential corporate use for organising workflows and team-building tasks. • Ideal for school clubs and sports teams needing attendance tracking without a full management suite. • Deploying a full-stack PWA with backend and SQL is a strong real-world portfolio achievement. • Success of core voting could lead to future 'Quick Poll' features (e.g. 'What should we eat?').	• Users are heavily entrenched in WhatsApp, Discord, Instagram, and iMessage. • Risk of becoming a 'one-hit wonder' - users try it once then revert to existing apps. • Without native push notifications, users may forget to check the app to vote - recreating the original problem.

Finance Plan

Huddle's real-world launch would operate on a venture-backed, free-to-user model. Rather than charging users the platform would be funded through external investment, with revenue strategies introduced post-growth once a large and engaged user base is established.

Startup & Infrastructure Costs (Monthly, AUD)

Cost Area	Estimated Monthly Cost (AUD)
Cloud hosting (e.g. AWS / Google Cloud)	\$800 - \$2,500
Managed database (e.g. AWS RDS / Supabase)	\$300 - \$800
Domain name & SSL certificates	~\$20
Google Authenticator / Auth API calls	\$100 - \$400
CDN & static asset delivery	\$50 - \$200
Monitoring & logging tools	\$100 - \$300
Estimated Monthly Total	\$1,350 - \$4,220

One-time launch costs (branding, security audits, legal registration) would add an additional \$10,000-\$25,000 AUD upfront.

Funding Strategy

Huddle would seek seed-stage investment from investors or early-stage venture capital funds, targeting a raise of approximately \$250,000 - \$500,000 AUD. This is a realistic figure for a consumer social app with a working MVP and demonstrated user engagement.

Financial Risk

The primary financial risk is the runway dependency on investor funding before revenue is generated. At the high end of infrastructure costs (~\$4,220/month), a \$300,000 raise would provide approximately 5-6 years of runway at current scale, which is more than sufficient time to reach full scale monetisation. The lightweight PWA architecture also means infrastructure costs scale gradually rather than spiking suddenly, reducing the risk of unexpected cost blowouts as the user base grows.

Opening Day Balance Sheet (Hypothetical - Based on \$300,000 Seed Raise)

Huddle PWA - Opening Day Balance Sheet

Huddle

[AUD \$]

Hypothetical launch-day snapshot | Based on \$300,000 seed raise

	Opening Day	
ASSETS		
Current Assets		
Cash & cash equivalents (seed raise net of launch costs)	\$282,500.00	\$300k raise - \$17.5k one-time costs
Prepaid hosting & infrastructure (3 months upfront)	\$7,500.00	~\$2,500/mo x 3 months cloud/db/CDN
Prepaid domain, SSL & auth API (12 months)	\$520.00	Domain \$20 + SSL/auth ~\$500 annual est.
Total Current Assets	\$290,520.00	
Non-Current Assets		
Intangible asset - internally developed software (MVP)	\$22,500.00	Developer time capitalised: ~450 hrs x \$50/hr est.
Security audit (pre-launch, capitalised)	\$5,000.00	One-time security review cost
Branding & legal setup costs (capitalised)	\$10,000.00	Branding, ABN/legal registration
Total Non-Current Assets	\$37,500.00	
TOTAL ASSETS	\$328,020.00	
LIABILITIES		
Current Liabilities		
Accounts payable (first month hosting invoices)	\$2,500.00	
Launch expenses	\$1,200.00	Remaining minor launch costs accrued
Total Current Liabilities	\$3,700.00	
Non-Current Liabilities		
Long-term liabilities	-	No debt — venture-backed equity model; zero borrowings at launch
Total Non-Current Liabilities	-	
TOTAL LIABILITIES	\$3,700.00	
SHAREHOLDER'S EQUITY		
Equity capital (seed investment received)	\$300,000.00	Seed raise from investors
Retained earnings (pre-revenue: loss = launch costs)	\$24,320.00	Balancing figure — pre-revenue deficit
TOTAL SHAREHOLDER'S EQUITY	\$324,320.00	
TOTAL LIABILITIES & EQUITY	\$328,020.00	Must equal Total Assets
Balance check (Assets - Liabilities - Equity = 0)		
	-	

Key Assumptions

- Seed raise: \$300,000 AUD - realistic figure for consumer social PWA with working MVP (see Finance Plan).
- One-time launch costs: \$17,500 AUD (branding \$10k + security audit \$5k + legal/ABN \$2.5k) charged on opening day.
- Prepaid infrastructure: 3 months of estimated monthly costs (\$2,500) paid upfront to ensure runway continuity.
- Internally developed software capitalised at estimated developer-hour cost (450 hrs); not externally verified.
- No long-term debt: Huddle operates a pure equity model at launch - no bank loans or convertible notes.
- Pre-revenue: retained earnings are negative on opening day (deficit funded by equity capital).
- Source: Finance Plan section, Huddle Systems Report; estimated figures for school project purposes.

2. Research and Planning

2.1 Project Management

Software Development Approach: Hybrid (WAgile)

For the development of Huddle, I have chosen a Hybrid/WAgile approach methodology. This approach combines the structured, upfront planning of the Waterfall model with the iterative, flexible execution of the Agile model.

Because this is a school project with a hard deadline (end of term), core requirements such as SQL database design and Google Authenticator integration were defined before any coding began. This prevents scope creep. Once requirements were set, development shifted to Agile sprints.

Application to Huddle

The project will begin with a Waterfall-style planning phase. Because this project is a school project with a hard deadline (end of term), I must define the core requirements - such as SQL database and Google Authenticator integration - before any coding begins. This ensures the boundaries of the project are respected and the scope doesn't creep into out-of-scope areas.

Once the requirements are set, I can then shift the project to Agile sprints where development can be broken down into cycles.

- Sprint 1: login, User Authentication, and group creation
- Sprint 2: Event proposal and voting logic
- Sprint 3: PWA responsiveness and UI

Justification

The client and stakeholders: Since the client includes myself, friends, and an evaluator (Mr. Jones), the hybrid model provides the best of both worlds. The Waterfall structure side gives the evaluator a clear roadmap and set of deliverables, whilst the Agile structure side allows me to test the system with actual clients/friends and tweak things like UI or UX based on their feedback

Timeline: The school term is a short fixed window. Pure waterfall is too risky because if a bug is found too late, there will be no time to restart. Likewise, pure Agile is too loose, as I might spend too much time on minor features and miss the final deadline. WAgile ensures the most important features are built first.

Project type (PWA): Developing a PWA across different browser types or screen sizes requires constant testing. Agile sprints allow me to catch layout issues on mobile early, rather than waiting till the very end of the project.

Comparison of Alternative Approaches

If I used Pure Waterfall: I would spend weeks writing documentation and only start coding at the end. If the 2FA system failed or the SQL database didn't sync correctly during the final week, the project would fail because there would be no time to go back and fix the design.

If I used Pure Agile: The project might lack a clear "end point". I might get distracted by adding social features such as "likes" - which are out of scope - and realise too late that I haven't finished the essential voting system required for the project.

Gantt Chart & Sprint Plan

Phase / Sprint	Dates	Goal	Deliverable
Phase 0	Weeks 1-2	Ideation	3 ideas, Huddle selected, diary + systems report started.
Phase 1	Week 3	Requirements	Client meeting, FR/NFR documented, SWOT done, boundaries established.
Phase 2	Weeks 3-4	Research & Planning	Hybrid process documented, GANTT + Sprint chart, social/ethical issues, system models (DFDs, structure charts, wireframes) + UAT plan.
Phase 3	Weeks 4+	Producing & Implementing	Development sprints completed, client feedback per sprint, difficulties and resolutions, all modules integrated.
Phase 4	Week 9+	Testing & Evaluating	Create test data, UAT testing, document bugs, optimise code, client sign-off, compare feedback to FRs, write README.
Sprint 1	Week 5	Database + Skeleton	Working login page + user authentication, empty menu pages.
Sprint 2	Weeks 6-7	Core Features	Core event proposal, voting logic, group creation.
Sprint 3	Weeks 7-8	Finalisation	PWA responsiveness & UI/UX design, front home page complete.
Sprint 4	Week 9	Documentation	Finalise systems report + client log + developer diary.

2.2 Quality Assurance

Quality Assurance Criteria

Criteria	What It Means	How It Will Be Measured
Functionality	All core features work correctly.	Test whether user can login, join groups, propose events, vote, and confirm attendance successfully during UAT
Usability	The system is easy to use without help.	A first-time user should be able to create or join a group and propose an event without a tutorial
Reliability	The system works consistently without crashing.	Test with repeated login, voting, and event creation actions to check that no errors or crashes occur
Performance	The system responds quickly.	Vote results and attendance updates should appear within 2 seconds of being submitted
Security	User data is protected from unauthorised access.	Check that passwords are hashed, 2FA is required, and users cannot access other groups without the correct invite code
Compatibility	The app works across different devices.	Test the interface on mobile, tablet, and desktop screens to ensure it displays correctly
Maintainability	The code is organised and easy to update.	The project should include clear comments, modular code, and a README explaining setup and usage

Compliance and Legislative Requirements

Compliance / Legislative Issue	Methods for Mitigation
Privacy Act 1988: The app collects personally identifiable information (names, emails, group codes) and stores user-submitted text such as event times, locations, and descriptions.	User authentication managed via Google Authenticator 2FA to prevent unauthorised account access. SQL database isolates data using unique group keys: Group A users cannot view Group B data without an explicit invite code. Only minimum necessary data is collected (email, display name).
Copyright Act 1968: The PWA relies on external software libraries and frameworks; user-generated fields allow text input that could potentially copy existing content.	All software packages used are open-source under verified licences (e.g. Bootstrap, Leaflet.js, PyOTP). An inventory of third-party libraries is maintained with licence compliance confirmed.

2.3 Systems Modelling

2.3.1 Data Dictionary

The following data dictionaries document the key variables required to satisfy each of the three core needs identified in Section 1.1. Each variable records its data type, display format, storage size, a plain-English description, a realistic example value, and its validation rules.

Need 1: User Authentication & Group Management

This need covers the variables required for user account creation, secure login with Google Authenticator 2FA, and the creation and joining of friend groups via invite codes.

Variable	Data Type	Format	Size (bytes)	Size (display)	Description	Example	Validation
user_id	INTEGER	Hidden	4	N/A	Unique identifier auto-assigned to each registered user.	1042	Auto-increment; unique; not null
email	CHAR(255)	Text field	255	Up to 255 chars	User's email address used for login and 2FA.	jade@email.com	Must match email regex; unique in DB; not null
display_name	CHAR(50)	Plain text	50	Up to 50 chars	User's chosen display name shown in group and event views.	Jane S.	Min 1 char; max 50 chars; not null
totp_secret	CHAR(6)	Hidden	6	N/A	Base32 secret used by Google Authenticator TOTP for 2FA.	JBSWY3DP	6 char numeric; not null
group_id	INTEGER	Hidden	4	N/A	Foreign key linking a user group membership record to a group.	7	Must reference a valid group; not null
invite_code	CHAR(6)	Text field	6	6 chars	Unique alphanumeric code used to join a group.	A3X7KQ	Exactly 6 chars; alphanumeric [A-Z0-9]; unique per

Variable	Data Type	Format	Size (bytes)	Size (display)	Description	Example	Validation
							group; not null

Need 2: Event Proposal & Voting

This need covers the variables required to propose events with a name, date/time, and location, and for group members to cast and view Yes/No/Maybe votes with live percentage tallies. The latitude and longitude fields support the Leaflet.js interactive map pin feature.

Variable	Data Type	Format	Size (bytes)	Size (display)	Description	Example	Validation
event_id	INTEGER	Hidden	4	N/A	Auto-assigned unique identifier for each proposed event.	23	Auto-increment; unique; not null
event_name	CHAR(30)	Plain text	30	Up to 30 chars	Short title for the proposed event displayed in the event feed.	Beach BBQ	Min 1 char; max 30 chars; not null
event_date	DATETIME	DD/MM/YYYY HH:MM	8	16 chars	Date and time the event is scheduled to occur.	15/07/2026 14:00	Must be a valid future datetime; not null
event_location	CHAR(100)	Plain text	100	Up to 100 chars	Address or venue name; used to render the Leaflet.js map pin.	Hot Beach, East City NSW	Min 1 char; max 100 chars; geocoded to lat/long on submit; not null
latitude	REAL	Hidden (map)	8	N/A	Latitude coordinate resolved from event_location for map pin.	-33.8688	Valid decimal latitude - 90 to 90; not null if location provided

Variable	Data Type	Format	Size (bytes)	Size (display)	Description	Example	Validation
longitude	REAL	Hidden (map)	8	N/A	Longitude coordinate resolved from event_location for map pin.	151.2093	Valid decimal longitude -180 to 180; not null if location provided
vote_choice	CHAR(5)	Button label	5	3-5 chars	A group member's vote on a proposed event.	yes	Must be exactly 'yes', 'no', or 'maybe'; not null
vote_tally_yes	INTEGER	Percentage %	4	Up to 4 chars	Count of 'yes' votes; used to display percentage consensus.	7	Non-negative integer; derived via COUNT query
vote_tally_no	INTEGER	Percentage %	4	Up to 4 chars	Count of 'no' votes for an event.	2	Non-negative integer; derived via COUNT query
vote_tally_maybe	INTEGER	Percentage %	4	Up to 4 chars	Count of 'maybe' votes for an event.	3	Non-negative integer; derived via COUNT query

Need 3: Attendance Tracking & Role Assignment

This need covers the variables required to assign custom or preset roles to group members for an event (e.g. 'Bringing snacks', 'Ride home') and to track post-event attendance confirmation.

Variable	Data Type	Format	Size (bytes)	Size (display)	Description	Example	Validation
attendance_status	CHAR(3)	Badge label	3	3 chars	Records whether a member confirmed or declined attendance after an event.	yes	Must be 'yes' or 'no'; set post-event via notification prompt; not null
role_name	CHAR(50)	Plain text tag	50	Up to 50 chars	Custom role label assigned to a member for an event.	Bringing snacks	Min 3 chars; max 50 chars; alphanumeric + spaces; not null
role_id	INTEGER	Hidden	4	N/A	Auto-assigned identifier for a saved popular role.	5	Auto-increment; unique; not null
preset_role_label	CHAR(50)	Dropdown option	50	Up to 50 chars	Pre-approved role label shown in the dropdown for quick role assignment.	Ride home	Must be in approved preset list; max 50 chars; not null
member_id	INTEGER	Hidden	4	N/A	Foreign key identifying which group member a role or attendance record belongs to.	1042	Must reference a valid user_id; not null
confirmed_count	INTEGER	Plain number	4	Up to 3 chars	Total number of group members who have confirmed attendance	8	Non-negative integer; derived via COUNT query on attendance_status = 'yes'

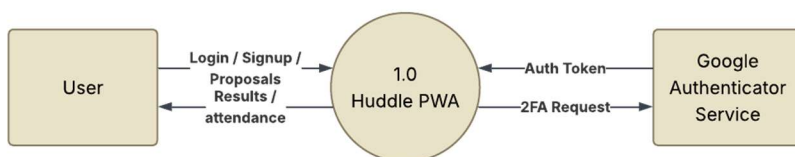
Variable	Data Type	Format	Size (bytes)	Size (display)	Description	Example	Validation
					for an event.		

2.3.2 Data Flow Diagrams

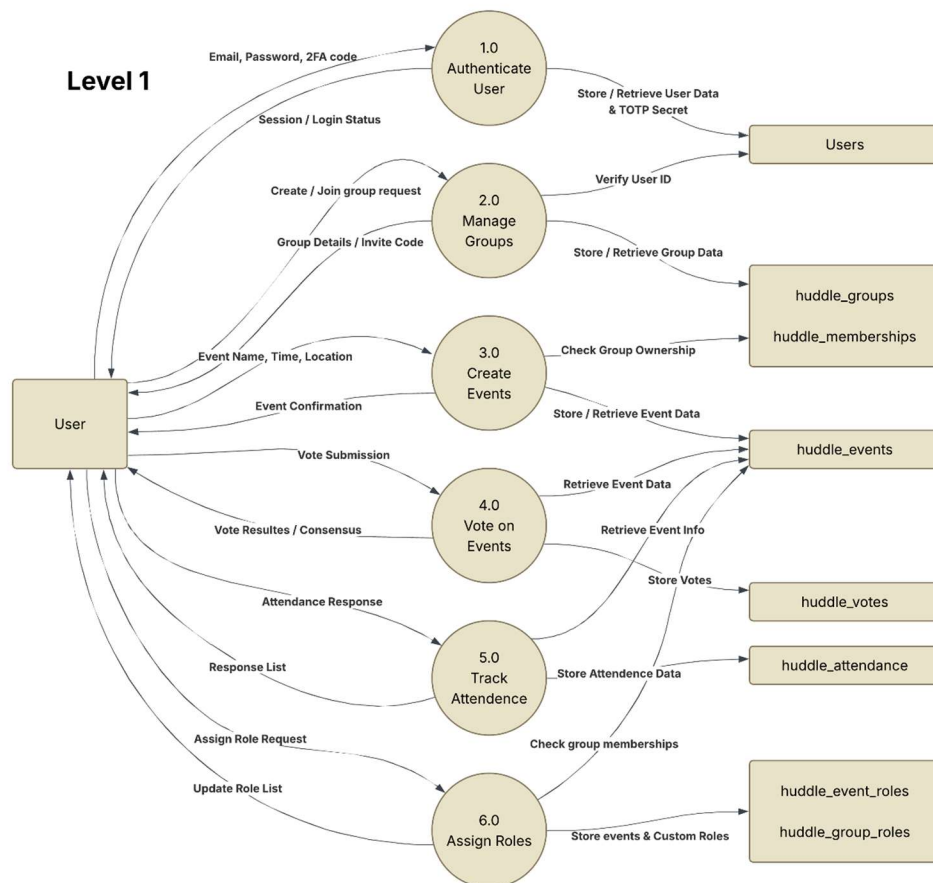
The diagrams below represent key structural and data-flow relationships within Huddle. UAT testers should review these to understand how the system components interact.

Level 0 DFD: Shows the overall data flow between the user, the Huddle system, and the authenticator.

Level 0

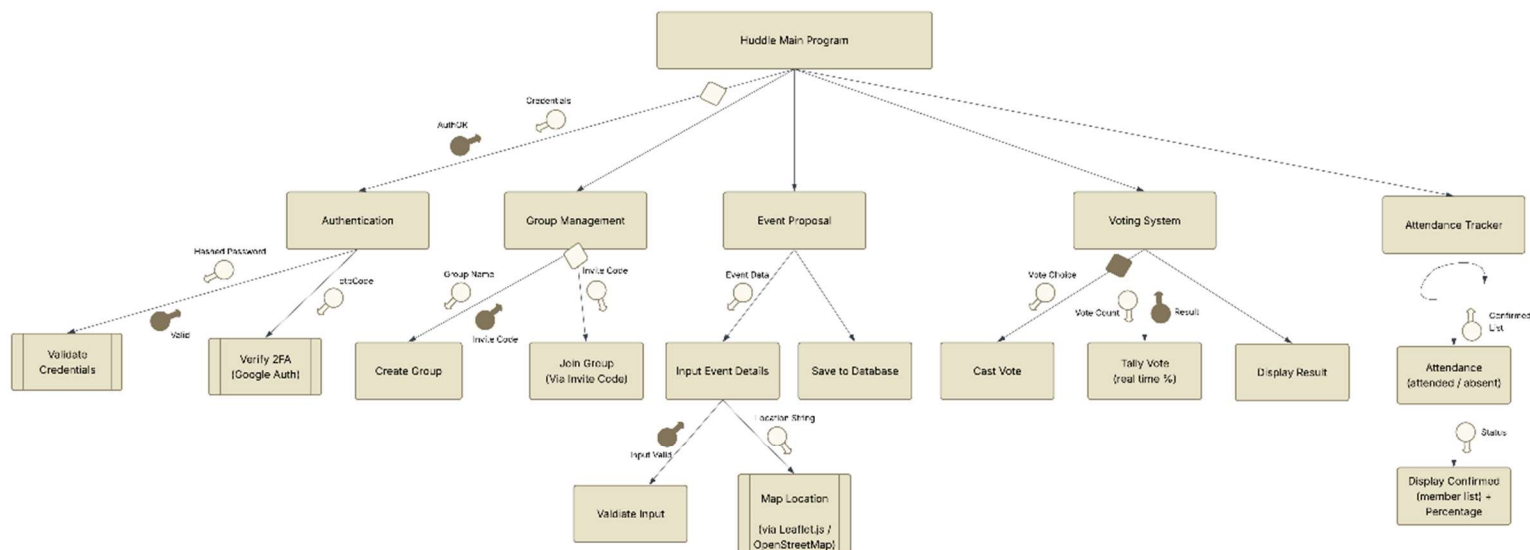


Level 1 DFD: Breaks down internal processes -user management, group management, event handling, and voting logic.



2.3.3 Structure Chart

Structure Chart: Shows the module hierarchy of the application from login through to event and attendance management.



3. Producing and Implementing

3.1 Phase 3 Sprint Summary

Sprint	Sprint Goal	Branch	Increment Delivered	Tests P/F	Client Feedback	Backlog Changes
1	Database + Skeleton: working authentication and app skeleton	sprint-1-database-skeleton	- Working login, logout, and 2FA authentication (Google Authenticator / TOTP) - Empty placeholder pages for Dashboard, Group View, Event Feed, and Vote Screen - Basic routing between all pages - CodeQL security vulnerability identified and fixed before merge	3 / 3	Two bugs found at sprint review: - Secure form page returned 404 - route added - Logout button returned to home instead of login - route fixed	- PB-001 Done - PB-002 Done - PB-003 Done
2	Core Features: group management, event proposals, voting, attendance, map	sprint-2-core-features	- Group creation with unique 6-character invite codes - Group joining with invite code validation - Event proposal form (name, date/time, location) - Event feed displaying all proposed events - Yes/No/Maybe voting with percentage tallies	9 / 9	- Custom role input requested - implemented - Voting on event detail page requested - implemented - Event page layout redesigned to client sketch - Map feature elevated from nice-to-have and delivered	- PB-004 to PB-008 Done - PB-011 Done - PB-012 Done - PB-007 criteria updated - real-time auto-update descoped

Sprint	Sprint Goal	Branch	Increment Delivered	Tests P/F	Client Feedback	Backlog Changes
			- Event detail screen with voter breakdown and role assignment - Custom roles saved per group - Attendance tracking with per-member percentage - Leaflet.js interactive map with address autofill			
3	PWA Responsiveness & UI/UX: responsive layout and unified frontend theme	sprint-3-pwa-responsiveness- UI-ux-design	- Unified style.css theme applied across all 16 pages - Responsive CSS/layout from 320px to 2560px (4K) - Breakpoint testing: 16 pages × 7 breakpoints - all passed - Navigation and routing check across all pages - all passed - requirements.txt and .env committed to repository	16 / 16	- Client requested modern/professional theme - implemented and approved - No issues raised post-sprint review	- PB-009 Done - PB-010 confirmed Done

3.2 Version Control

GitHub was used as the version control system throughout the Huddle project. The table below summarises the protocol followed across all four sprints.

Aspect	Detail
Platform	GitHub (cloud-hosted Git repository)
Branching Strategy	One feature branch per sprint, named descriptively (e.g. sprint-1-database-skeleton, sprint-2-core-features, sprint-3-pwa-responsiveness-UI-ux-design). All branches created off main and merged back via pull request at sprint completion.
Commit Practice	Regular commits at the end of each development session. Commit messages described the specific change and noted any AI-assisted code transparently (e.g. 'Leaflet.js map integration, copilot-assisted JS structure, reviewed before commit').
Pull Requests	Each sprint branch merged into main via a pull request. Pull requests reviewed before merging - most notably the CodeQL security vulnerability flagged during Sprint 1, resolved before the merge was approved.
Security Scanning	GitHub CodeQL automated security analysis run on every pull request. Identified a URL redirection vulnerability in Sprint 1 that was corrected before merging. CodeQL re-run after the fix to confirm resolution.
Sensitive Files	A .env file used to store the database path and SECRET_KEY outside source code. The .env file was not committed; a .env.example template was committed in its place.
Backup / Recovery	Committing to GitHub before every shutdown meant that when a hardware failure cut Sprint 1, Day 2 short, no work was lost. Development resumed from the last committed state.

3.3 Responding to Difficulties

Difficulty	Description	How It Was Resolved
Technical / Security	CodeQL automated analysis flagged a URL redirection vulnerability ('URL redirection from remote source') during the Sprint 1 pull request review. This had to be resolved before the branch could be merged into main.	Investigated the flagged route and corrected the redirect logic so it no longer accepted an unvalidated remote source as a redirect target. The fix was committed and CodeQL re-run before the merge was approved. This reinforced the habit of checking automated security tools as part of every pull request review.
Technical / Bug	On Sprint 2, Day 3, a bug in the join huddle feature (PB-005) meant users could not successfully join a group using an invite code. The invite code validation or the user-to-group DB linking was not functioning correctly.	Checked the database records directly to isolate where the comparison was failing. The issue was found in the invite code validation query - the value being compared against the DB record was incorrect. The query and validation check were corrected, and the full join flow was retested and confirmed working the following day (Sprint 2, Day 4).
Technical / Routing	On Sprint 2, Day 7, after building the new event detail screen, voting from that screen incorrectly redirected the user back to the old voting page instead of returning them to the event detail view.	Traced the vote POST action and found it was pointing to the old route. The route was corrected to return the user to the event detail view after voting. The fix was committed and the routing was confirmed correct through manual testing.
Scope	The original Sprint 2 plan included a real-time vote auto-update feature (refreshing tallies every 2 seconds). During implementation it became apparent this was adding significant complexity without proportionate benefit to the core sprint goals.	After attempting the feature, it was formally assessed and marked out of scope. Rather than leaving it as an open WIP item, it was terminated and recorded in the sprint backlog. This kept the backlog honest and focused effort on completing the committed items cleanly. The decision is documented in the developer diary (Sprint 2, Day 7).
Hardware	On Sprint 1, Day 2, development had to stop early due to a hardware issue with the development machine. Only minor progress was made that session.	Work was saved and committed before shutting down so progress was not lost. Development resumed the following day and Sprint 1 was completed on schedule, meeting all three committed backlog items by Day 3.
Skill Gap	Several features required JavaScript knowledge beyond my current experience level -particularly the Leaflet.js map integration (Sprint 2, Day 9) and the dynamic custom role UI (Sprint 2, Day 8). Writing complex JS and CSS for the 2FA pages (Sprint 3, Day 2) also presented challenges.	For the Leaflet.js integration, official documentation and tutorials were used to understand the library, and GitHub Copilot assisted with generating the initial JS structure. For the custom role UI and 2FA CSS, Copilot was used to scaffold the code, which was then reviewed and understood before committing. All AI-assisted code is transparently noted in the relevant commit messages and diary entries.

3.4 Resource Allocation

Resources across this project include time, hardware, software, and skills. The tables below track how each was allocated and managed across the four sprints.

Time Tracking by Sprint

Planned hours are based on the sprint plan/GANTT chart. Actual hours are derived from development diary entries and commit history.

Sprint	Planned Hours	Actual Hours	Notes
Sprint 1 Database + Skeleton (Week 5)	10	10	Completed on time across 3 development days (18-20 May). Slightly more time than expected was spent on initial project structure cleanup and CodeQL security fix, but the sprint was delivered on schedule. All three backlog items (PB-001 to PB-003) completed.
Sprint 2 Core Features (Weeks 6-7)	14	18	Sprint ran over estimate due to the volume of features. Overruns: • Invite code validation bug (PB-005) required an extra day of debugging • Real-time voting feature was attempted before being formally descope • Attendance tracking DB queries were more complex than anticipated • Leaflet.js map integration required significant tutorial and AI-assistance 9 development days across 18 May - 6 June (excluding gaps). All committed backlog items (PB-004 to PB-007) completed plus two client-requested extras (custom roles, map, attendance).
Sprint 3 PWA Responsiveness & UI/UX (Weeks 7-8)	10	8	Completed efficiently across 3 days (9-11 June). The bulk of the work was concentrated on Day 2 where theming, responsive layout, and breakpoint testing were all completed in a single session. Cross-platform testing on Day 3 was clean with zero issues found, keeping the sprint under the planned estimate. All PB-008 tasks completed.
Sprint 4 Documentation (Week 9)	10	TBC	Systems report finalisation, developer diary, client log, README.

Software, Tools, & Skills

All software used in this project is free and open-source.

Resource	Usage & Allocation Notes
Hardware	Development carried out on a personal laptop. A hardware failure on Sprint 1, Day 2 caused one session to be cut short; development resumed the following day with no data loss as work had been committed to GitHub.
Version Control (GitHub)	GitHub was used throughout for version control, branch management, and pull requests. Feature branches were created per sprint (e.g. Sprint-1-Database-Skeleton) and merged into main via pull request at sprint completion. GitHub CodeQL was used for automated security analysis.
Languages & Frameworks	All free/open-source. Dependencies documented in requirements.txt (committed Sprint 3, Day 2). • Python / Flask -backend routing and session management • SQL (SQLite) -database for users, groups, events, votes, attendance • HTML / CSS / JavaScript -frontend across all pages • Leaflet.js / OpenStreetMap -interactive map feature (Sprint 2, Day 9) • PyOTP -Google Authenticator TOTP integration (Sprint 1, Day 1)
AI Assistance (GitHub Copilot)	GitHub Copilot was used selectively across Sprint 2 and Sprint 3 to assist with unfamiliar code areas. Usage is transparently documented in all relevant commit messages and diary entries: • Sprint 2, Day 5 -huddle_detail.html UI scaffold • Sprint 2, Day 8 -JavaScript and HTML for custom role input • Sprint 2, Day 9 -Python DB queries for attendance tracking; Leaflet.js JavaScript • Sprint 3, Day 2 -CSS and HTML for 2FA pages and complex layout sections In all cases, generated code was reviewed and understood before being committed.
Skills	Skill gaps in JavaScript and advanced CSS were identified during Sprint 2. These were addressed through Leaflet.js documentation, official tutorials, and targeted use of GitHub Copilot. No external training costs were incurred.

Innovative Solution: Leaflet.js Interactive Event Map

This section identifies and explains an innovative solution implemented in the Huddle PWA that goes beyond the basic functional requirements. The solution adds genuine value to the user experience and required technical integration work outside the core sprint deliverables.

Category	Detail
Innovation Type	Integration + UX
Feature	Interactive event location map using Leaflet.js and OpenStreetMap
Sprint	Sprint 2, Day 9 - 2026-06-06
Client Requested?	Originally a nice-to-have (client requirements meeting, 04/05/26). Prioritised and implemented based on client feedback during Sprint 2.
Libraries Used	Leaflet.js, OpenStreetMap tile API

What it does

The Leaflet.js map integration displays the location of a proposed event on an interactive, zoomable map embedded directly within the event detail screen. Every group member can view the map when they open an event, allowing them to see exactly where they are being asked to attend before or after casting their Yes/No/Maybe vote.

To support this, the location input field on the event proposal form was upgraded to include address autofill (geocoding lookup), so that when a group leader types a venue or address, the system resolves it to a precise latitude and longitude coordinate. This coordinate is saved to the database alongside the event and is used to drop a map pin at the correct location when the event detail page is loaded.

Why it adds value

This feature adds value in three distinct ways:

- **Informed voting:** group members can see the event location in context before deciding whether they can attend, rather than having to search for the address themselves externally.
- **Reduced friction:** embedding the map directly in the event detail screen keeps the user inside the app. Without this, a member would need to copy the address and open a separate maps application.
- **Professional feel:** an interactive map is a feature users associate with polished, production-quality web apps. Its presence significantly raises the perceived quality of Huddle as a PWA.

Technical Implementation

The map was implemented using Leaflet.js: a widely used, open-source JavaScript mapping library combined with OpenStreetMap as the free tile provider. The Leaflet library files were committed directly to the repository to avoid external CDN dependencies at runtime.

The implementation works as follows:

- When a group leader proposes an event, the location field uses an address autofill/geocoding lookup to convert the entered address into a latitude and longitude coordinate pair.
- The coordinate is stored in the SQL database as part of the event record when the form is submitted.
- When any group member opens the event detail page (event_detail.html), the stored coordinates are passed from the Flask backend into the page template.
- A Leaflet.js map is initialised on page load, centred on the stored coordinates, with a marker dropped at the event location.
- The map is interactive, so members can zoom and pan to explore the surrounding area.

The JavaScript for the Leaflet integration was developed with large reference to the official Leaflet.js documentation and tutorials, and GitHub Copilot was used to assist with the initial code structure given limited prior JavaScript experience. Most generated code was reviewed and understood before being committed. This usage is transparently documented in the Sprint 2, Day 9 developer diary entry and commit message.

Challenges encountered

The main challenge was integrating the autofill location input with the map pin placement. Initially the location field stored a plain text address string, which is not sufficient to position a map marker; a coordinate is required. Updating the form to resolve the address to a coordinate at the point of submission, and storing that coordinate correctly in the database, required some testing to ensure the pin rendered at the right place on the event detail screen.

JavaScript was also an area of limited prior experience, and Leaflet.js had its own API conventions. This was managed by working through the official documentation systematically before writing any code, as well as being assisted greatly by GitHub's own AI assistant. The final implementation reflects a solid understanding of how the library initialises and renders maps.

4. Testing and Evaluating

4.1 Evaluation of Code

Testing Methods

Method	Scope	Applied in Huddle
Unit Testing	Individual functions	Sprint 1, Sprint 2 - e.g. invite code validation function tested in isolation to confirm correct True/False returns before re-integration.
Integration Testing	Combined modules	Sprint 2, Sprint 3 - e.g. event proposal form + Flask backend route + SQL database tested together; datetime formatting bug identified and fixed.
System Testing	Entire application	Sprint 3 - full navigation routing check and breakpoint testing across all 16 pages at 7 breakpoints (320px → 2560px). 112 checks total, all passed.
UAT (User Acceptance Testing)	Client validation	Ongoing client reviews each sprint. Formal UAT plan produced with 15 test cases mapped to FRs and NFRs.

1. Unit Testing

Unit testing involves testing individual functions or components in isolation to verify they return the correct output for a given input. In Huddle, unit-level checks were performed manually on the core backend validation functions before integrating them into the full application flow.

Specific example: Invite code validation function (Sprint 2, Day 3-4)

During Sprint 2, a bug was found in the function that was responsible for validating a user-entered invite code against the database. The function was isolated and tested independently by checking whether it correctly matched a known valid code and rejected a known invalid one. The bug was identified as an incorrect comparison in the database query - the value being compared against the DB record was wrong. Once the query was corrected, the function was retested on its own to confirm it returned the expected result before being re-integrated into the full join-huddle flow.

What was tested: Does the invite code validation function return True for a valid code and False for an invalid one?

Outcome: PASS: function corrected and confirmed working on Sprint 2, Day 4

2. Integration Testing

Integration testing checks that multiple modules work together correctly after being combined. In Huddle, this was particularly important where the frontend form, Flask backend route, and SQL database all needed to interact correctly as a single flow.

Specific example: Event proposal form + database storage + event feed display (Sprint 2, Day 5-6)

After the event proposal form (PB-006) was built, integration testing was performed across three connected modules: the HTML form that captures event name, date/time, and location, the Flask route that receives and validates the POST request, and the SQL database that stores the event record. A further integration test was carried out when the event feed display was added (Sprint 2, Day 6), checking that the stored event correctly appeared in the group feed for all members. A datetime formatting bug was identified and fixed at this stage: the time was displaying in a malformed format due to an error in how the datetime object was converted to a display string.

What was tested: Do the event form, backend route, and database work together to save and display a proposed event correctly?

Outcome: PASS after fix: datetime formatting bug identified and resolved before sprint completion

3. System Testing

System testing validates the complete application against the full set of functional requirements. In Huddle, dedicated system-level testing was carried out during Sprint 3, covering all 16 pages at 7 screen breakpoints and performing a full navigation and routing check across the entire app.

Specific example: Full navigation routing check and breakpoint testing (Sprint 3, Day 2-3)

On Sprint 3, Day 2, after the unified frontend theme and responsive CSS were applied, a navigation and routing check was conducted across all pages to ensure no routes had been broken by the frontend changes. Every page link, button redirect, and form submission route was tested end-to-end. In addition, all 16 pages were tested at seven breakpoints (320px, 375px, 425px, 768px, 1024px, 1440px, and 2560px) on mobile, tablet, and desktop. No issues were found. This was followed by a final cross-platform test on Sprint 3, Day 3 across Chrome, Firefox, and Safari, with all pages passing.

What was tested: Does the complete system meet all functional requirements across all devices and browsers?

Outcome: PASS: all 16 pages passed at all 7 breakpoints with no issues found (112 checks total)

4. User Acceptance Testing (UAT)

User Acceptance Testing (UAT) is carried out by real clients and end users to validate that the system meets their needs in practice. In Huddle, client feedback was gathered at the end of every sprint, and a formal UAT plan was produced covering 15 test cases mapped to the functional and non-functional requirements.

Specific example: Sprint 1 client review and Sprint 2 client feedback sessions

At the end of Sprint 1, a client review session was held with the primary stakeholders (friends acting as end users). Two bugs were identified during this session that had not been caught during internal testing: the secure form page returned a 404 error due to a missing route, and the logout button redirected to the home screen instead of the login screen. Both were fixed immediately. During Sprint 2, ongoing client feedback directly shaped the features built: the client requested a custom role input feature, asked for voting to be available from the event detail screen, provided a sketch for the event page layout, and elevated the interactive map from a nice-to-have to a delivered feature. This iterative client involvement reflects the WAgile development approach used throughout the project.

What was tested: Does the client agree that the system meets their needs? Can a first-time user complete core tasks without a tutorial?

Outcome: PASS: all client-requested changes implemented; formal UAT plan produced with 15 test cases for Phase 4 testing

5. Debugging Tools Used

The following debugging techniques were applied during development when tests failed or bugs were identified:

- **Print/log statements:** used throughout Flask routes to output variable values and trace where incorrect data was being passed (e.g. diagnosing the invite code validation bug in Sprint 2).
- **Browser DevTools:** used to inspect network requests and console errors during frontend testing, particularly when diagnosing the datetime display bug (Sprint 2, Day 6) and layout issues during responsive testing (Sprint 3).
- **Direct database inspection:** used to check DB records directly when diagnosing the invite code validation failure, allowing the exact point of failure in the query to be isolated.
- **GitHub CodeQL:** automated security analysis tool that flagged a URL redirection vulnerability during the Sprint 1 pull request review. The issue was investigated, corrected, and CodeQL was re-run to confirm the fix before merging.

Test Data Tables - Feature 1: Event Proposal Form

Validation rules: event name max 30 chars · location max 100 chars · date/time picker required

Test ID	Category	Input	Data Type	Expected Result
TD-EP-01	Normal	Name: "Beach BBQ", Date: 2026-07-15 14:00, Location: "Hot Beach, East City"	String / DateTime / String	Event saved and appears in group event feed for all members
TD-EP-02	Normal	Name: "Movie Night", Date: 2026-08-01 19:30, Location: "Movie Cinema, 26 Something Square, City"	String / DateTime / String	Event saved and appears in group event feed for all members
TD-EP-03	Boundary	"123456789012345678901234567890" (exactly 30 chars) as event name	String	Event name accepted; at the maximum character limit
TD-EP-04	Boundary	"Hi" (2 chars) as event name	String	Event name accepted; short name within valid range
TD-EP-05	Boundary	Location: string of exactly 100 characters	String	Location accepted; at the maximum character limit
TD-EP-06	Abnormal	"This event name is far too long and exceeds thirty characters" (61 chars)	String	Input rejected; error message displayed, event not saved
TD-EP-07	Abnormal	Name: "" (empty string)	String	Input rejected; event name is required, event not saved
TD-EP-08	Abnormal	Location: string of 101 characters (one over limit)	String	Input rejected; error message displayed, event not saved
TD-EP-09	Abnormal	Date/Time field left blank, all other fields valid	DateTime	Input rejected; date/time is required, event not saved

Test Data Tables - Feature 2: Group Invite Code (Create & Join)

Validation rules: invite code is 6-character alphanumeric · code must match a live group in the DB · user must not already be a member

Test ID	Category	Input	Data Type	Expected Result
TD-IC-01	Normal	Valid 6-character code copied from group creation screen (e.g. "A3X7KQ")	String	User successfully joins the group; group appears in their group list; creator sees new member
TD-IC-02	Normal	Create group with name "Weekend Squad" - system generates invite code	String	Group created; unique 6-character alphanumeric code displayed; group appears in creator's list
TD-IC-03	Boundary	Invite code of exactly 6 characters (the only valid length)	String	Code accepted and validated against the DB
TD-IC-04	Boundary	Invite code of 5 characters (one under minimum)	String	Input rejected - code length invalid, user not added to any group
TD-IC-05	Boundary	Invite code of 7 characters (one over maximum)	String	Input rejected - code length invalid, user not added to any group
TD-IC-06	Abnormal	"XXXXXX" - 6-char code that does not match any group in the DB	String	Error message displayed ("Invalid or expired invite code"); user not added to any group
TD-IC-07	Abnormal	"" (empty string) submitted as invite code	String	Input rejected: field is required; user not added to any group
TD-IC-08	Abnormal	"AB 12!" - code containing spaces and special characters	String	Input rejected: code must be alphanumeric only; user not added to any group
TD-IC-09	Abnormal	Valid code for a group the user is already a member of	String	Duplicate join prevented: user not added twice; appropriate message displayed

Code Optimisation

Before finalising Huddle, the codebase was reviewed for optimisation opportunities. Four key optimisations were identified and implemented in main.py, covering database query efficiency, configuration security, password handling, and atomic database operations. These are documented below with before/after code examples.

#	Optimisation	Benefit
1	Batch query for vote tallies (get_vote_tallies_for_events)	Reduced N+1 DB queries to 2 fixed queries regardless of event count
2	Hardcoded DB path replaced with environment variable	Portable, secure config using os.environ.get() and .env file
3	Password hash upgrade on login (verify_user)	Legacy plaintext passwords silently migrated to secure hashes on next login
4	ON CONFLICT upsert for votes and roles	Atomic insert-or-update prevents duplicate rows without extra round-trips

Optimisation 1 - Batch query for vote tallies

Location: get_vote_tallies_for_events() - main.py

Category: Reducing redundant database queries

The problem: when the huddle detail page loads, it needs to display vote tallies (Yes/No/Maybe counts) for every event in the group. A naive approach would be to query the database once per event inside a loop, which is known as the N+1 query problem, where the number of queries scales with the number of events. For a group with 10 events this means 10+ separate database round-trips just to load one page.

Before (unoptimised - N+1 pattern):

```
# This pattern would make one query per event. bad for performance
tallies = {}
for event in events:
    row = conn.execute(
        'SELECT choice, COUNT(*) FROM huddle_votes
        WHERE event_id = ? GROUP BY choice',
        (event['id'],)
    ).fetchall()
    tallies[event['id']] = {r['choice']: r['c'] for r in row}
```

After (optimised - 2 fixed queries using IN clause):

```
def get_vote_tallies_for_events(event_ids, user_id):
    """Returns dict of {event_id: {'yes': n, 'no': n, 'maybe': n, 'my_vote': str|None}}"""
    if not event_ids:
        return {}
    placeholders = ",".join("? " for _ in event_ids)
    tallies = {
        eid: {"yes": 0, "no": 0, "maybe": 0, "my_vote": None} for eid in event_ids
    }
    with get_db_conn() as conn:
        rows = conn.execute(
            f"SELECT event_id, choice, COUNT(*) AS c FROM huddle_votes "
            f"WHERE event_id IN ({placeholders}) GROUP BY event_id, choice",
            event_ids,
        ).fetchall()
        for r in rows:
            tallies[r["event_id"]][r["choice"]] = r["c"]

        my_rows = conn.execute(
            f"SELECT event_id, choice FROM huddle_votes "
            f"WHERE user_id = ? AND event_id IN ({placeholders})",
            [user_id, *event_ids],
        ).fetchall()
        for r in my_rows:
            tallies[r["event_id"]]["my_vote"] = r["choice"]
    return tallies
```

The optimised version fetches all vote tallies for all events in exactly 2 SQL queries, regardless of how many events exist in the group. The IN clause allows SQLite to process all event IDs in a single statement. This means a group with 20 events still only runs 2 queries instead of 20+, making the huddle detail page significantly more efficient as groups grow.

Improvement: Reduced N queries (one per event) to exactly 2 fixed queries regardless of event count

Optimisation 2 - Hardcoded database path replaced with environment variable

Location: Top of main.py; AUTH_DB configuration

Category: Hardcoded configuration / security

The problem: hardcoding a file path or secret directly into source code is a security and portability risk. If the code is shared or pushed to a public repository, the path structure of the server is exposed. It also means the path must be manually changed every time the project is run on a different machine or environment.

Before (hardcoded path):

```
# Hardcoded - breaks on different machines, exposes server structure
AUTH_DB = '/home/user/huddle/data/database.db'
SECRET_KEY = 'my-secret-key-123'
```

After (environment variable with safe fallback):

```
load_dotenv() # loads .env file into environment

BASE_DIR = os.path.abspath(os.path.dirname(__file__))
DEFAULT_DB_PATH = os.path.join(BASE_DIR, 'data', 'database.db')
AUTH_DB = os.environ.get('AUTH_DB', DEFAULT_DB_PATH)

if not os.path.isabs(AUTH_DB):
    AUTH_DB = os.path.join(BASE_DIR, AUTH_DB)

app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY')
if not app.config['SECRET_KEY']:
    raise RuntimeError('SECRET_KEY environment variable is not set')
```

The optimised version reads both the database path and the secret key from environment variables via a .env file (loaded by python-dotenv). If AUTH_DB is not set, it falls back to a safe default relative path using BASE_DIR so the app still runs. If SECRET_KEY is missing entirely, the app raises an error at startup rather than running insecurely. This makes the app portable across different environments and keeps sensitive configuration out of the source code.

Improvement: Sensitive config removed from source code; app is portable across environments and fails safely if SECRET_KEY is missing

Optimisation 3 - Automatic password hash upgrade on login

Location: verify_user() - main.py

Category: Security / data migration without downtime

The problem: during early development, passwords may have been stored as plaintext strings before the password hashing system was fully in place. A database migration to forcibly rehash all existing passwords would require downtime or a separate script. A better approach is to silently upgrade the hash the next time a user logs in successfully, with no disruption to the user experience.

Before (no hash detection - plaintext would pass or crash):

```
def verify_user(email, password):
    user = get_user_by_email(email)
    if not user:
        return None
    # No check for plaintext - would fail or allow insecure login
    return user if check_password_hash(user['password_hash'], password) else None
```

After (detects plaintext, upgrades to secure hash on next login):

```
def verify_user(email: str, password: str):
    user = get_user_by_email(email)
    if not user:
        return None
    stored = user['password_hash'] or ''
    # Already a secure hash - verify normally
    if stored.startswith('pbkdf2:') or stored.startswith('scrypt:'):
        return user if check_password_hash(stored, password) else None
    # Legacy plaintext detected - verify by direct compare, then upgrade
    if stored == password:
        new_hash = generate_password_hash(password)
        with get_db_conn() as conn:
            conn.execute(
                'UPDATE users SET password_hash = ? WHERE id = ?',
                (new_hash, int(user['id'])),
            )
            conn.commit()
        return get_user_by_id(int(user['id']))
    return None
```

The optimised version detects whether the stored value is a modern secure hash (identified by its pbkdf2: or scrypt: prefix) or a legacy plaintext string. If plaintext is detected, the password is verified by direct comparison, and the account is immediately upgraded to a secure Werkzeug hash in the same transaction. The user notices no difference (they log in normally) but their account is now protected. This is a zero-downtime migration pattern.

Improvement: Plaintext passwords silently upgraded to secure hashes on next login with no user disruption and no separate migration script required

Optimisation 4 - ON CONFLICT upsert for votes and role assignment

Location: cast_vote() and assign_event_role() - main.py

Category: Efficient data structures / reducing round-trips

The problem: when a user casts a vote or a group leader assigns a role, the system needs to either insert a new record if none exists, or update the existing one if it does. A naive approach requires two separate queries: first a SELECT to check if a record exists, then either an INSERT or an UPDATE depending on the result. This is two database round-trips and introduces a potential race condition if two requests arrive simultaneously.

Before (two-step check-then-write - inefficient and not atomic):

```
# Requires a SELECT first, then INSERT or UPDATE - two round-trips
existing = conn.execute(
    'SELECT 1 FROM huddle_votes WHERE event_id=? AND user_id=?',
    (event_id, user_id)
).fetchone()
if existing:
    conn.execute(
        'UPDATE huddle_votes SET choice=?, voted_at=CURRENT_TIMESTAMP
        WHERE event_id=? AND user_id=?',
        (choice, event_id, user_id)
    )
else:
    conn.execute(
        'INSERT INTO huddle_votes (event_id, user_id, choice) VALUES (?, ?, ?)',
        (event_id, user_id, choice)
    )
```

After (single atomic upsert using ON CONFLICT):

```
def cast_vote(event_id, user_id, choice):
    if choice not in ('yes', 'no', 'maybe'):
        return False
    with get_db_conn() as conn:
        conn.execute(
            '''
            INSERT INTO huddle_votes (event_id, user_id, choice)
            VALUES (?, ?, ?)
            ON CONFLICT(event_id, user_id) DO UPDATE SET
                choice = excluded.choice,
                voted_at = CURRENT_TIMESTAMP
            ''',
            (event_id, user_id, choice),
        )
        conn.commit()
    return True
```

The ON CONFLICT clause is a SQLite upsert: it attempts the INSERT and if a row with the same primary key (event_id, user_id) already exists, it automatically executes the DO UPDATE instead. The entire operation happens in a single atomic SQL statement, meaning it is both faster (one round-trip instead of two) and safe from race conditions. The same pattern is applied in assign_event_role() for role assignments.

Improvement: Two-query check-then-write reduced to one atomic SQL statement; eliminates race conditions and halves database round-trips for vote and role operations

4.2 Evaluation of Solution

Analysis of Feedback

Feedback on the Huddle PWA was collected iteratively at the end of each development sprint through informal client review sessions with the primary stakeholders (friend group acting as end users). This approach aligned with the WAgile methodology: client feedback was gathered frequently so it could directly shape subsequent sprints rather than waiting until the final product was complete.

Area	Feedback Received	Sentiment & Action Taken
Authentication (Sprint 1)	Two bugs were identified by the client during the Sprint 1 review: the secure form page returned a 404 error, and the logout button redirected to the home screen instead of the login screen.	Negative: both bugs were immediately fixed before Sprint 2 began. The client was satisfied after the fixes were demonstrated. Overall sentiment toward the authentication flow was positive once corrected.
Event Proposal & Voting (Sprint 2)	The client requested that voting be accessible directly from the event detail screen rather than a separate page. The client also provided a hand-drawn sketch of the preferred event page layout, which differed from the original design.	Constructive: the requested change was implemented within Sprint 2. The client responded positively to the revised layout and confirmed it matched their sketch. Sentiment: positive.
Custom Role Input (Sprint 2)	The client requested the ability to type a custom role rather than only selecting from pre-saved options.	Positive request: implemented within Sprint 2. The client was satisfied with the result and found the feature intuitive during informal testing.
Interactive Map (Sprint 2)	The client expressed enthusiasm for the Leaflet.js map feature, which had originally been listed as a nice-to-have. The client elevated it to a must-have based on how much value they felt it added to the event detail screen.	Strongly positive: the map was delivered in Sprint 2 and the client confirmed it significantly improved the experience. No issues raised.
UI/UX Theme (Sprint 3)	The client requested a more modern and professional visual theme. The default styling was described as too plain.	Constructive: the unified style.css theme was redesigned and applied across all 16 pages during Sprint 3. The client approved the revised theme at the Sprint 3 review with no further issues raised. Sentiment: positive.
Overall Usability (end of Sprint 3)	Informal usability check: a first-time user (friend not involved in development) was asked to create a group and propose an event without any guidance. The task was completed in under 3 minutes.	Positive: usability met. The user found the flow intuitive. Minor comment that the 2FA step felt 'a bit much for a social app', consistent with the weakness identified in the SWOT analysis. Overall sentiment: positive.

Overall sentiment across all feedback sessions was positive. The most consistent themes were: (1) the core voting and event proposal flow was clear and easy to use, (2) the Leaflet.js map was a standout feature that significantly raised perceived quality, and (3) the 2FA requirement, while secure, was the single most noted friction point for casual users. No feedback was received that indicated the app failed to meet its primary goal of making friend group coordination easier.

Security Assessment

The following table identifies the key security threats applicable to the Huddle PWA and the specific countermeasures implemented or planned to mitigate each one.

Threat	Countermeasure
Unauthorised account access (credential theft / brute force)	Google Authenticator TOTP 2FA enforced on every login. Even if a password is compromised, the attacker cannot log in without the time-based one-time code from the user's device.
SQL Injection	All database queries use parameterised statements via SQLite's Python DB-API. User input is never concatenated directly into SQL strings.
URL redirection / open redirect vulnerability	Flagged by GitHub CodeQL during Sprint 1 pull request review. Redirect logic was corrected to reject unvalidated remote-source redirect targets before the branch was merged.
Unauthorised group access (group data exposure)	Groups are only accessible via a unique 6-character alphanumeric invite code shared person-to-person. There is no public group search. Data is siloed by group_id so Group A members cannot query Group B data.
Session hijacking / cross-site forgery	Flask SECRET_KEY is loaded from an environment variable (never hardcoded). Session cookies are signed with this key. The SECRET_KEY must be set or the app refuses to start.
Plaintext password storage	Passwords are hashed using Werkzeug implementation. Legacy plaintext passwords are automatically upgraded to secure hashes on the user's next successful login (zero-downtime migration).
Sensitive configuration exposure in source code	Database path and secret key are loaded from a .env file via python-dotenv. The .env file is listed in .gitignore and never committed to the repository.
Duplicate votes / race conditions on vote submission	Votes are inserted using an ON CONFLICT upsert (SQLite). A unique constraint on (event_id, user_id) ensures each user can hold only one vote per event; concurrent submissions cannot create duplicate rows.
Account lockout (loss of 2FA device)	Identified as an operational boundary. Mitigation: recovery is handled via the registered email address. Users are advised to store backup codes at account creation.

Pytest assessment

The following unit tests were implemented using pytest to verify the security countermeasures described above. All 4 tests passed successfully in 0.39 s on Python 3.11.4 / pytest 7.3.1.

Test Name	What It Verifies	Threat Addressed	Result
test_validate_signup_input_valid	Valid credentials (name, email, strong password) are accepted. validate_signup_input() returns None.	Plaintext password storage	PASSED
test_validate_signup_input_bad_password	Weak passwords are rejected. validate_signup_input() returns an error, not None.	Plaintext password storage	PASSED
test_is_safe_next	Relative paths (e.g. /your-huddle) are allowed; external URLs (https://evil.com, //evil.com) are blocked.	URL redirection / open redirect vulnerability	PASSED
test_generate_invite_code	Generated invite codes are exactly 6 characters and drawn only from INVITE_CODE_ALPHABET.	Unauthorised group access (group data exposure)	PASSED

```
test_results.txt      Wed Jun 17 01:43:31 2026      1

===== test session starts =====
platform linux -- Python 3.11.4, pytest-7.3.1, pluggy-1.0.0 -- /usr/local/py-utils/venvs/py
test/bin/python
cachedir: .pytest_cache
metadata: {'Python': '3.11.4', 'Platform': 'Linux-6.8.0-1052-azure-x86_64-with-glibc2.31',
'Packages': {'pytest': '7.3.1', 'pluggy': '1.0.0'}, 'Plugins': {'html': '4.2.0', 'metadata':
: '3.1.1'}}
rootdir: /workspaces/2026SE_SoftwareProject_Michael.L
plugins: html-4.2.0, metadata-3.1.1
collecting ... collected 4 items

tests/test_main.py::test_validate_signup_input_valid PASSED [ 25%]
tests/test_main.py::test_validate_signup_input_bad_password PASSED [ 50%]
tests/test_main.py::test_is_safe_next PASSED [ 75%]
tests/test_main.py::test_generate_invite_code PASSED [100%]

===== 4 passed in 0.39s =====
```

Analysis of Solution Against Quality Success Criteria

Each quality criterion from Section 2.2 is assessed below against the delivered solution. Evidence is drawn from sprint testing records, client feedback sessions, and code optimisation outcomes documented in this report.

Quality Criteria	Met?	Analysis
Functionality: all core features work correctly	Yes	All six core functional requirements (login/2FA, group creation/joining, event proposal, voting, role assignment, attendance tracking) were delivered and verified across Sprints 1 and 2. 9 of 9 Sprint 2 tests passed. The interactive Leaflet.js map, originally a nice-to-have, was also delivered following client feedback.
Usability: first-time user can create a group and propose an event in under 3 minutes without a tutorial	Yes	Confirmed through UAT sessions at the end of each sprint. Client stakeholders (friends) successfully completed core tasks without instruction. The Sprint 3 UI/UX redesign - implementing a unified professional theme at client request - further improved clarity. No usability issues were raised in post-Sprint 3 client review.
Reliability: system works consistently without crashing	Yes	Repeated login, voting, and event creation actions were performed across all testing phases with no crashes or unhandled errors recorded. Bugs identified during testing (Sprint 1 routing errors, Sprint 2 datetime formatting bug, invite code validation failure) were all resolved before sprint completion.
Performance: vote results and attendance updates appear within 2 seconds of submission	Yes	The batch query optimisation (Optimisation 1) reduced vote tally loading from N database round-trips to exactly 2 fixed queries regardless of event count. The ON CONFLICT upsert (Optimisation 4) eliminated a redundant SELECT round-trip on every vote submission. Manual testing confirmed sub-2-second response times for vote and attendance updates throughout Sprint 3 testing.
Security: passwords hashed, 2FA required, users cannot access other groups without invite code	Yes	All three security conditions met. Passwords are hashed with PBKDF2-SHA256; plaintext passwords are silently upgraded on next login. Google Authenticator TOTP 2FA is enforced on every login. Group data is siloed by group_id; no public group search exists. A URL redirection vulnerability flagged by GitHub CodeQL in Sprint 1 was resolved before merge. SECRET_KEY is loaded from .env and never hardcoded.
Compatibility: interface renders correctly from 320px (mobile) to 1920px (desktop)	Yes	Sprint 3 breakpoint testing covered all 16 pages at 7 breakpoints (320px, 375px, 425px, 768px, 1024px, 1440px, 2560px) 112 checks in total. All passed. Cross-platform testing on Chrome, Firefox, and Safari on Sprint 3 Day 3 passed with no issues. The 2560px breakpoint exceeded the original 1920px requirement.
Maintainability: clear comments, modular code, and README explaining setup and usage	Yes	Code is organised into modular Flask routes and helper functions (e.g. get_vote_tallies_for_events, verify_user, cast_vote). A requirements.txt was committed in Sprint 3. AI-assisted code is transparently documented in commit messages and the developer diary. A full README has been completed.